



Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«Московский государственный технический университет  
имени Н.Э. Баумана  
(национальный исследовательский университет)»  
(МГТУ им. Н.Э. Баумана)

---

ФАКУЛЬТЕТ ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»  
КАФЕДРА ИУ7 «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

---

## КУРСОВАЯ РАБОТА

*НА ТЕМУ:*

*Разработка базы данных для хранения и  
обработки данных платформы игровых  
модификаций*

|                                 |          |                 |                |
|---------------------------------|----------|-----------------|----------------|
| Студент                         | ИУ7-65Б  |                 | Колоколов Г.И. |
|                                 | (группа) | (подпись, дата) | (И.О. Фамилия) |
| Руководитель курсовой<br>работы |          | (подпись, дата) | Шибанова Д.А.  |
|                                 |          |                 | (И.О. Фамилия) |
| Консультант                     |          | (подпись, дата) | (И.О. Фамилия) |

2026 г.

# РЕФЕРАТ

Расчетно-пояснительная записка 40 с., 7 рис., 16 таблица, 8 источников, 2 приложения.

БАЗА ДАННЫХ, POSTGRESQL, ИГРОВЫЕ МОДИФИКАЦИИ, РЕЛЯЦИОННАЯ МОДЕЛЬ ДАННЫХ, ИНДЕКСИРОВАНИЕ, РОЛЕВАЯ МОДЕЛЬ, REDIS.

Расчетно-пояснительная записка посвящена разработке базы данных для платформы игровых модификаций. Целью работы является создание структуры хранения данных, обеспечивающей управление, поиск и хранение модификаций, а также поддержку разграничения прав доступа пользователей.

В ходе выполнения работы был проведен анализ предметной области, выделены основные сущности и их взаимосвязи, рассмотрены существующие аналоги и сформулированы требования к разрабатываемой системе. Выполнено сравнение различных моделей хранения данных и обоснован выбор реляционной модели в качестве основного решения.

На этапе проектирования разработана структура базы данных, построена диаграмма сущность–связь и схема отношений, выполнена нормализация данных до третьей нормальной формы. Также спроектирована ролевая модель доступа и разработан алгоритм работы триггера, обеспечивающего соблюдение бизнес-логики системы.

В технологической части база данных реализована средствами PostgreSQL. Созданы таблицы, ограничения целостности, индексы и триггеры. Для повышения производительности системы предусмотрено использование Redis в качестве кэширующего хранилища. Проведено тестирование разработанных механизмов.

В исследовательской части выполнен анализ влияния различных типов индексов PostgreSQL на производительность поиска. Исследованы индексы B-Tree, Hash, GIN, GiST и BRIN. По результатам исследования установлено, что индекс B-Tree является предпочтительным решением для поиска модификаций, поскольку использует наименьшее количество памяти и сопоставим по скорости с Hash индексом. Индекс BRIN обеспечивает наилучшие результаты среди других индексов при выполнении диапазонных запросов.

# СОДЕРЖАНИЕ

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>ВВЕДЕНИЕ</b>                                                            | <b>7</b>  |
| <b>1 Аналитическая часть</b>                                               | <b>8</b>  |
| 1.1 Сущности предметной области                                            | 8         |
| 1.2 Сравнение аналогичных решений                                          | 8         |
| 1.3 Требования, предъявляемые к разрабатываемому сервису                   | 9         |
| 1.4 Диаграмма прецедентов                                                  | 10        |
| 1.5 Диаграмма «сущность—связь»                                             | 11        |
| 1.6 Анализ моделей хранения данных                                         | 11        |
| 1.6.1 Дореляционные модели данных                                          | 11        |
| 1.6.2 Реляционная модель данных                                            | 12        |
| 1.6.3 Постреляционные модели хранения данных                               | 13        |
| 1.6.4 Сравнительный анализ моделей данных                                  | 14        |
| 1.6.5 Обоснование выбора модели                                            | 14        |
| 1.7 Вывод                                                                  | 15        |
| <b>2 Конструкторская часть</b>                                             | <b>16</b> |
| 2.1 Диаграмма базы данных                                                  | 16        |
| 2.2 Ролевая модель на уровне базы данных                                   | 16        |
| 2.3 Анализ функциональных зависимостей и проверка третьей нормальной формы | 17        |
| 2.4 Разработка алгоритма работы триггера                                   | 21        |
| 2.5 Вывод                                                                  | 22        |
| <b>3 Технологическая часть</b>                                             | <b>23</b> |
| 3.1 Выбор средств реализации                                               | 23        |
| 3.2 Реализация базы данных                                                 | 23        |
| 3.3 Ограничения целостности                                                | 24        |
| 3.4 Реализация триггеров                                                   | 24        |
| 3.5 Ролевая модель                                                         | 25        |
| 3.6 Кэширование                                                            | 26        |
| 3.7 Тестирование                                                           | 26        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 3.8      | Вывод . . . . .                                              | 27        |
| <b>4</b> | <b>Исследовательская часть . . . . .</b>                     | <b>28</b> |
| 4.1      | Цель исследования . . . . .                                  | 28        |
| 4.2      | Описание исследования . . . . .                              | 28        |
| 4.3      | Сравнение размеров индексов . . . . .                        | 29        |
| 4.4      | Исследование скорости поиска по точному совпадению . . . . . | 30        |
| 4.5      | Исследование диапазонного поиска . . . . .                   | 31        |
| 4.6      | Вывод . . . . .                                              | 32        |
|          | <b>ЗАКЛЮЧЕНИЕ . . . . .</b>                                  | <b>33</b> |
|          | <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .</b>            | <b>34</b> |
|          | <b>Приложение А . . . . .</b>                                | <b>35</b> |
|          | <b>Приложение Б . . . . .</b>                                | <b>36</b> |

# ВВЕДЕНИЕ

Современная игровая индустрия активно развивается, и значительную роль в ее эволюции играют игровые модификации (моды). Моды позволяют пользователю изменять игровые механики, визуальные эффекты, сюжет и игровой процесс, расширяя возможности базовых игр. В связи с этим возникает потребность в специализированных платформах, которые обеспечивали бы удобное управление, хранение и распространение модификаций, а также поддерживали взаимодействие между авторами контента и конечными пользователями.

Цель работы — разработка базы данных для платформы игровых модификаций, обеспечивающей хранение, управление и поиск модификаций, а также поддержку ролевой модели доступа на уровне БД.

Для достижения цели работы были поставлены следующие задачи:

- провести анализ предметной области;
- спроектировать базу данных;
- разработать базу данных для хранения и обработки данных модификаций и связанных с ними сущностей;
- исследовать характеристики разработанной базы данных.

# 1 Аналитическая часть

## 1.1 Сущности предметной области

Основными сущностями предметной области являются:

- игра — программа для которой создаются модификации;
- категория — категория, к которой может относиться модификация;
- модификация — набор различных версий модификаций;
- версия модификации — определенная версия некоторой модификации;
- файл модификации — определенный файл некоторой версии модификации;
- пользователь — человек, использующий платформу;
- комментарий — текст, который пользователь оставляет к модификации;
- подборка — множество сгруппированных пользователем модификаций.

## 1.2 Сравнение аналогичных решений

Критерии сравнения:

- К1 — возможность скачивать модификации без регистрации;
- К2 — возможность создания пользовательских наборов модификаций;
- К3 — наличие обратной связи у создателей модификаций (комментарии).

Сравнение было приведено в таблице 1.1.

Таблица 1.1 — Таблица сравнения аналогичных сервисов

| Сервис     | К1 | К2 | К3 |
|------------|----|----|----|
| Nexus Mods | —  | +  | +  |
| CurseForge | +  | —  | +  |
| Modrinth   | +  | +  | —  |

При сравнении аналогичных решений было выявлено, что ни один из рассмотренных сервисов не обладает всеми выбранными функциональными возможностями одновременно, поэтому разрабатываемая система должна объеди-

нять их в рамках единой платформы.

### **1.3 Требования, предъявляемые к разрабатываемому сервису**

#### **Акторы**

- гость — неавторизованный пользователь, который может просматривать и скачивать модификации других пользователей;
- авторизованный пользователь — аналогично гостю, но может выкладывать свои модификации и оставлять отзывы на модификации других пользователей.
- администратор — занимается модерацией выложенных пользователями модификаций, изменение справочной информации.

#### **Функциональные требования**

Необходимо предоставить возможность:

- пользователям выкладывать собственные модификации и оставлять комментарии на чужих модификациях;
- скачивать модификации, в том числе неавторизованным пользователям;
- пользователям составлять собственные подборки модификаций;
- администрации проводить модерацию выложенных модификаций;
- производить поиск модификаций с помощью различных фильтров.

#### **Ограничения**

- максимальный размер одного файла — 100 МБ;
- максимальное количество файлов для одной версии модификации — 5;
- максимальный размер изображения модификации — 3 МБ;
- допустимые форматы для изображений — png, svg, jpg, jpeg;
- модификация обязательно должна относиться к какой-либо игре;
- у модификации обязательно должна быть хотя бы одна версия.

## 1.4 Диаграмма прецедентов

В соответствии с выделенными акторами и предъявленными требованиями была составлена диаграмма прецедентов платформы. Диаграмма представлена на рисунке 1.1. Она отражает основные сценарии взаимодействия пользователей с системой и демонстрирует ключевые функциональные возможности платформы. На основе данной диаграммы были определены основные требования к реализации пользовательского и административного функционала.

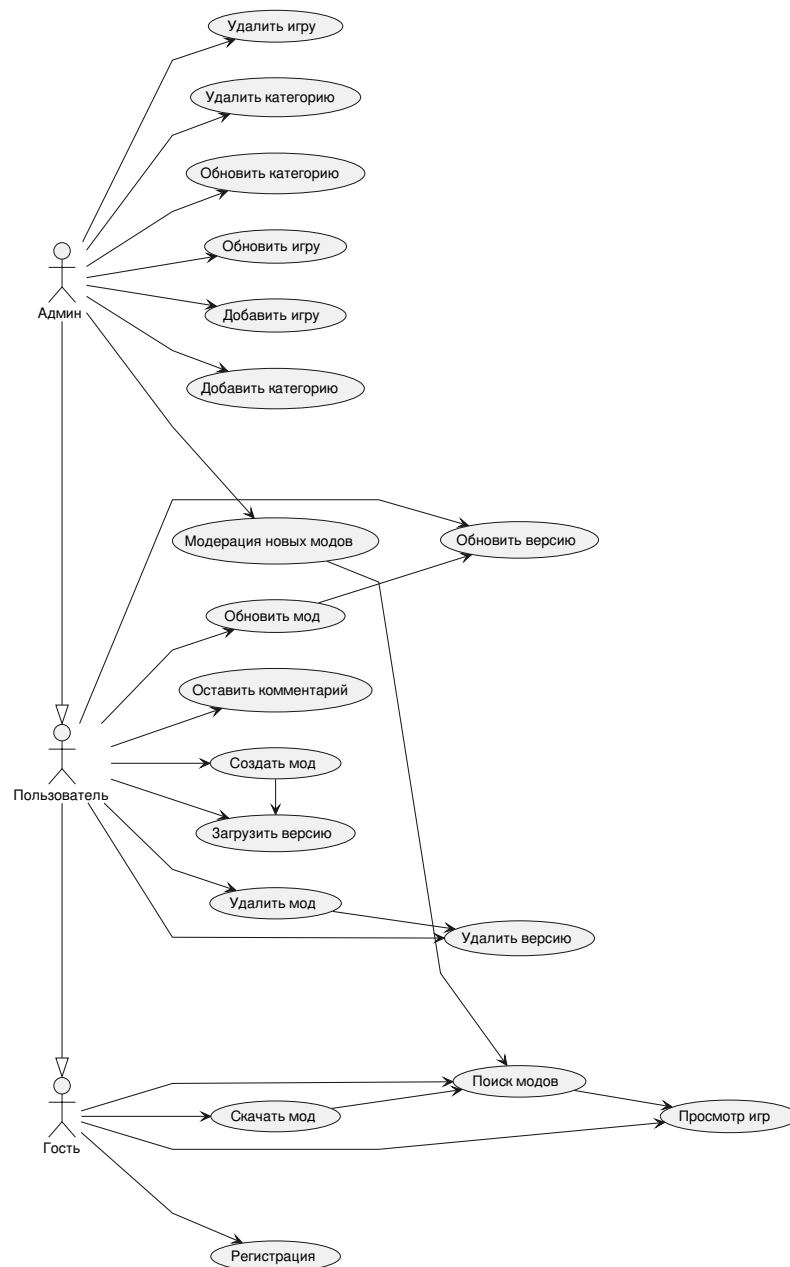


Рисунок 1.1 — Диаграмма прецедентов



## 1.5 Диаграмма «сущность–связь»

Диаграмма «сущность–связь» в нотации Чена необходима для концептуального моделирования предметной области до перехода к конкретной базе данных [1]. Сделанная на основе анализа предметной области диаграмма представлена на рисунке 1.2. Она отражает основные сущности системы, их атрибуты и взаимосвязи между ними. Полученная модель служит основой для дальнейшего проектирования структуры базы данных и её логической схемы.

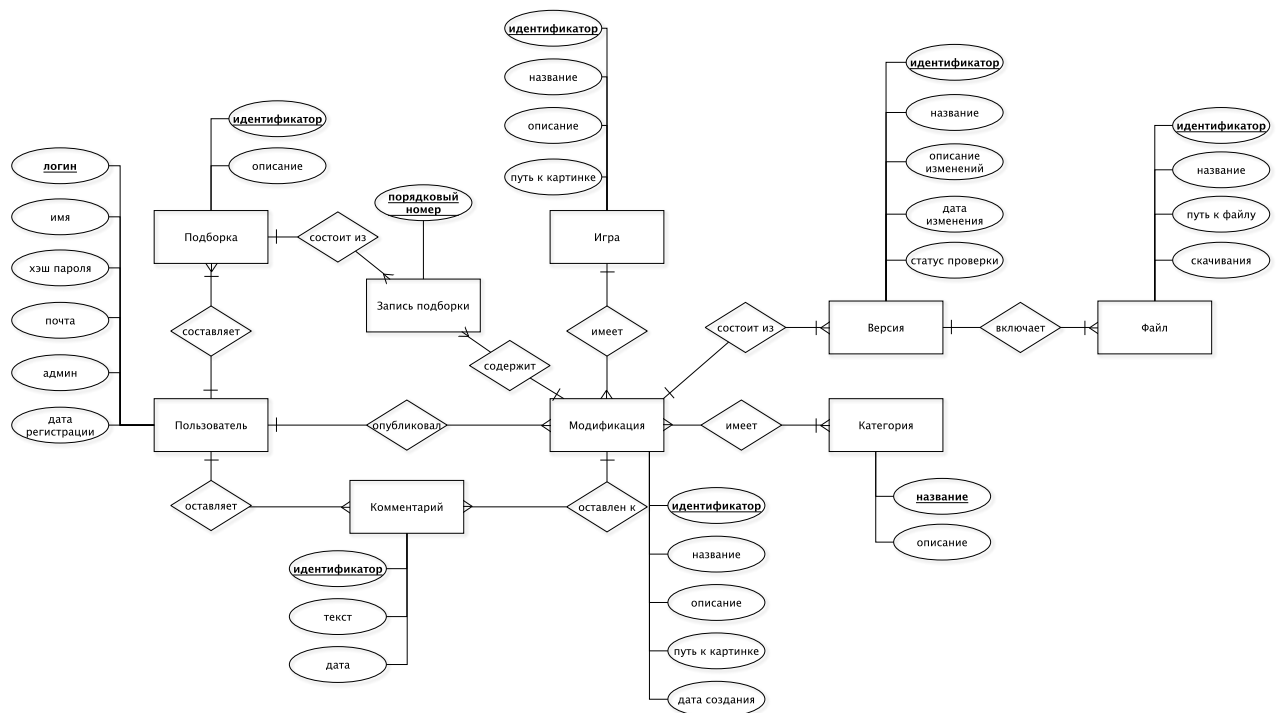


Рисунок 1.2 — Модель «сущность–связь»

## 1.6 Анализ моделей хранения данных

Модель данных представляет собой логическую структуру, определяющую правила организации, хранения и манипулирования данными в информационной системе [2].

### 1.6.1 Дореляционные модели данных

К дореляционным моделям относятся иерархическая и сетевая модели. Характерной чертой данных моделей является навигационный принцип доступа к данным, при котором логика перемещения по записям реализуется на уровне прикладного программного обеспечения [2].

## Иерархическая модель

Иерархическая модель строится по принципу древовидного графа. Объекты связаны отношениями типа «один ко многим» ( $1 : N$ ), где каждый «потомок» может иметь только одного «предка».

- к преимуществам модели относится высокая скорость доступа к данным при поиске по вертикальным связям за счет использования физических указателей;
- к недостаткам следует отнести избыточность данных при попытке реализации связей типа «многие ко многим» ( $M : N$ ), а также жесткую зависимость данных: удаление родительского объекта влечет за собой автоматическое удаление всех подчиненных записей [2].

## Сетевая модель

Сетевая модель является расширением иерархической и допускает наличие у одного объекта нескольких «предков», что позволяет напрямую описывать отношения типа  $M : N$ .

- преимуществом модели является отсутствие дублирования данных, характерного для иерархического подхода;
- основной недостаток заключается в высокой сложности схемы и трудоемкости разработки, обусловленной необходимостью явного описания путей навигации по связям-указателям между записями [2].

### 1.6.2 Реляционная модель данных

Реляционная модель, предложенная Э. Коддом, базируется на математической теории множеств и логике предикатов. Данные представляются в виде совокупности нормализованных отношений (таблиц) [2].

В отличие от навигационных моделей, реляционный подход использует декларативный язык запросов SQL. К ключевым характеристикам модели, согласно правилам Кодда, относятся:

- целостность сущностей и ссылочная целостность, реализуемые через механизмы первичных и внешних ключей;
- логическая и физическая независимость данных: возможность измене-

ния структуры хранения или описания данных без переработки прикладного ПО;

— минимизация избыточности: за счет приведения таблиц к нормальным формам реляционная модель исключает аномалии вставки, обновления и удаления данных [2–4].

### **1.6.3 Постреляционные модели хранения данных**

Парадигма NoSQL ориентирована на решение задач горизонтального масштабирования и обработки неструктурированных данных. В отличие от реляционной модели в постреляционных моделях используется денормализация для повышения скорости доступа к данным [2].

#### **Модель «ключ-значение»**

Данная модель представляет собой ассоциативное хранилище, где доступ к данным осуществляется по уникальному идентификатору (ключу).

— основным достоинством является минимальное время отклика, не зависящее от объема хранимой информации;

— ограничением модели является отсутствие механизмов контроля связей и невозможность выполнения выборок по содержимому хранимых значений [2, 5].

#### **Документная модель**

Модель «ключ-значение», где значением является документ. В рамках данной модели данные хранятся в виде структурированных документов (например, в формате JSON, CSV и т.д.), обладающих собственной внутренней иерархией.

— преимуществом модели является гибкость схемы данных и возможность хранения связанных объектов внутри одного документа;

— к недостаткам относится низкая скорость выполнения операций объединения (JOIN) между различными коллекциями, что требует реализации логики связывания на уровне приложения [2, 5].

### 1.6.4 Сравнительный анализ моделей данных

Для объективного выбора архитектуры хранения данных необходимо провести сравнение рассмотренных моделей по ключевым техническим критериям (K1–K5):

- K1 — обеспечение строгой ссылочной целостности на уровне СУБД;
- K2 — наличие декларативного языка запросов (SQL и аналоги);
- K3 — поддержка связей типа «многие ко многим» ( $M : N$ ) без дублирования данных;
- K4 — отсутствие аномалий вставки, обновления и удаления за счет механизмов нормализации;
- K5 — возможность горизонтального масштабирования системы.

Результаты сравнительного анализа представлены в таблице 1.2.

Таблица 1.2 — Сравнительная характеристика моделей хранения данных

| Модель данных | K1    | K2    | K3 | K4 | K5    |
|---------------|-------|-------|----|----|-------|
| Иерархическая | —     | —     | —  | —  | —     |
| Сетевая       | —     | —     | +  | —  | —     |
| Реляционная   | +     | +     | +  | +  | + / — |
| Ключ-значение | —     | —     | —  | —  | +     |
| Документная   | + / — | + / — | —  | —  | +     |

### 1.6.5 Обоснование выбора модели

Проведенный анализ предметной области выявил наличие множества взаимосвязанных сущностей (пользователи, модификации, файлы) с преобладанием связей типа  $M : N$ . Применение дореляционных моделей нецелесообразно ввиду высокой сложности поддержки навигационных путей доступа. Использование документных NoSQL-моделей приведет к нарушению целостности данных из-за отсутствия механизмов связывания объектов.

На основании изложенного, для реализации основного хранилища системы была выбрана реляционная модель данных, поскольку она обеспечивает требуемый уровень ссылочной целостности.

Для оптимизации производительности при обработке часто запрашиваемых данных (кэширование сессий, самых частых запросов) необходимо использовать вспомогательное хранилище типа «ключ–значение». Данный комбини-

рованный подход позволяет сочетать транзакционную надежность реляционной СУБД с высокой скоростью доступа NoSQL-решения.

## **1.7 Вывод**

В аналитической части проанализированы различные модели хранения данных, а также построена диаграмма «сущность–связь» предметной области в нотации Чена. В результате анализа было принято решение использовать реляционную модель для основного хранения данных платформы модификаций, так как она удобно описывает связи между играми, модификациями, версиями, файлами и пользователями, а также обеспечивает поддержку сложных запросов и ссылочной целостности.

Дополнительно к реляционной модели планируется использовать хранилище типа «ключ–значение» для кэширования часто запрашиваемых данных. Это позволит снизить нагрузку на основную базу и ускорить отклик сервиса.

## 2 Конструкторская часть

### 2.1 Диаграмма базы данных

На основе анализа предметной области была спроектирована реляционная схема базы данных. Разработанная структура обеспечивает хранение информации о пользователях, игровых проектах, модификациях, версиях и файлах, а также поддерживает систему категорий, комментариев и пользовательских коллекций.

Диаграмма базы данных представлена на рисунке 2.1.

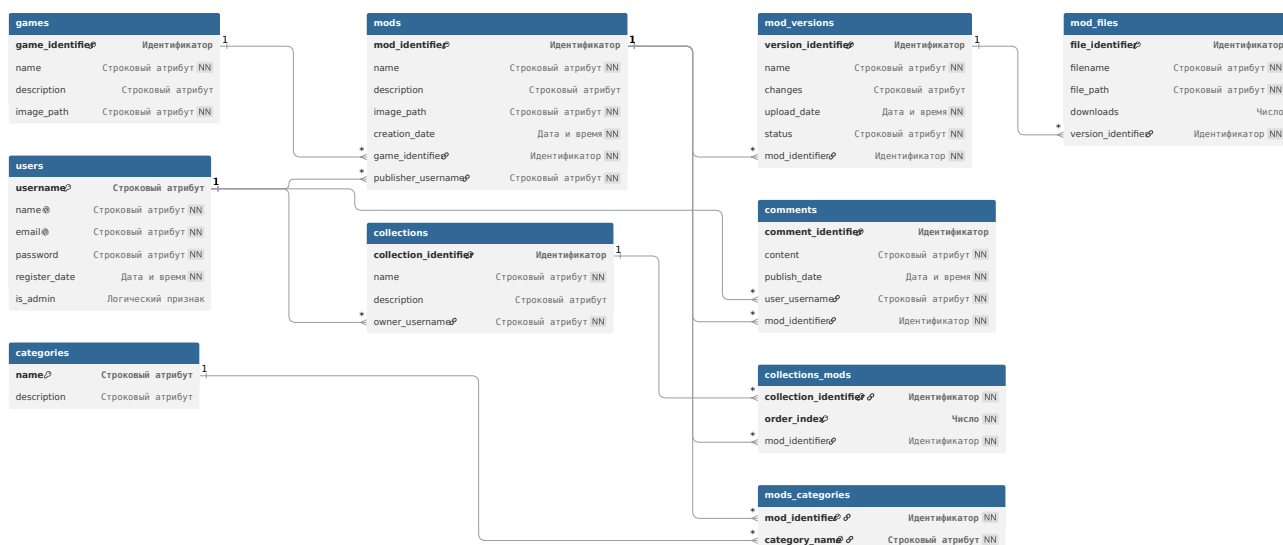


Рисунок 2.1 — Диаграмма базы данных

### 2.2 Ролевая модель на уровне базы данных

Для разграничения прав доступа в системе выделены три роли. Матрица доступа к таблицам представлена в таблице 2.1. В таблице были использованы следующие обозначения:

- C (Create) — создание;
- R (Read) — чтение;
- U (Update) — обновление;
- D (Delete) — удаление.

Таблица 2.1 — Ролевая модель доступа к объектам БД

| Таблица          | Гость | Пользователь | Администратор |
|------------------|-------|--------------|---------------|
| users            | R     | RU           | CRUD          |
| games            | R     | R            | CRUD          |
| mods             | R     | CRUD         | CRUD          |
| mod_versions     | R     | CRUD         | CRUD          |
| mod_files        | R     | CRD          | CRUD          |
| categories       | R     | R            | CRUD          |
| mods_categories  | R     | CRD          | CRUD          |
| comments         | R     | CRD          | CRUD          |
| collections      | R     | CRUD         | CRUD          |
| collections_mods | R     | CRUD         | CRUD          |

## 2.3 Анализ функциональных зависимостей и проверка третьей нормальной формы

Для исключения избыточности и аномалий обновления проведена нормализация схемы. Ниже приведены описания атрибутов отношений, выделены потенциальные ключи и проведен анализ функциональных зависимостей (ФЗ) на соответствие условиям третьей нормальной формы (3НФ).

### Отношение users

Таблица 2.2 — Атрибуты отношения users

| Атрибут       | Тип          | Назначение                    |
|---------------|--------------|-------------------------------|
| username (PK) | Строка       | Уникальный логин пользователя |
| name          | Строка       | Отображаемое имя              |
| email         | Строка       | Электронная почта             |
| password      | Строка       | Хэш пароля                    |
| register_date | Дата и время | Дата регистрации              |
| is_admin      | Логический   | Признак администратора        |

Потенциальные ключи:  $K_1 = \{username\}$ ,  $K_2 = \{email\}$ ,  $K_3 = \{name\}$ .

$$\Phi\mathcal{Z}: F = \begin{cases} username \rightarrow \{name, email, password, register\_date, is\_admin\}; \\ email \rightarrow \{username, name, password, register\_date, is\_admin\}; \\ name \rightarrow \{username, email, password, register\_date, is\_admin\}. \end{cases}$$

*Вывод:* все детерминанты ФЗ являются потенциальными ключами. Отношение в 3НФ.

### Отношение games

Таблица 2.3 — Атрибуты отношения games

| Атрибут     | Тип    | Назначение         |
|-------------|--------|--------------------|
| id (PK)     | Строка | Идентификатор игры |
| name        | Строка | Название игры      |
| description | Текст  | Описание проекта   |
| image_path  | Строка | Путь к обложке     |

*Потенциальный ключ:*  $K = \{id\}$ .

*ФЗ:*  $F = \{id \rightarrow \{name, description, image\_path\}\}$

*Вывод:* неключевые атрибуты зависят только от ключа. Отношение в 3НФ.

### Отношение mods

Таблица 2.4 — Атрибуты отношения mods

| Атрибут           | Тип          | Назначение                |
|-------------------|--------------|---------------------------|
| id (PK)           | Строка       | Идентификатор модификации |
| name              | Строка       | Название модификации      |
| description       | Текст        | Описание                  |
| image_path        | Строка       | Путь к обложке            |
| creation_date     | Дата и время | Дата добавления           |
| game_id (FK)      | Строка       | Ссылка на игру            |
| publisher_id (FK) | Строка       | Ссылка на автора          |

*Потенциальный ключ:*  $K = \{id\}$ .

*ФЗ:*  $F = \left\{ id \rightarrow \begin{matrix} \{name, description, image\_path, creation\_date, game\_id, \\ publisher\_id\} \end{matrix} \right\}$

*Вывод:* транзитивные зависимости не обнаружены. Отношение в 3НФ.



## Отношение mod\_versions

Таблица 2.5 — Атрибуты отношения mod\_versions

| Атрибут     | Тип                                    | Назначение           |
|-------------|----------------------------------------|----------------------|
| id (PK)     | Универсальный уникальный идентификатор | Идентификатор версии |
| name        | Строка                                 | Название версии      |
| changes     | Текст                                  | Список изменений     |
| upload_date | Дата и время                           | Дата загрузки        |
| status      | Перечисление                           | Статус проверки      |
| mod_id (FK) | Строка                                 | Ссылка на мод        |

Потенциальные ключи:  $K_1 = \{id\}$ ,  $K_2 = \{name, mod\_id\}$ .

$$\Phi 3: F = \begin{cases} id \rightarrow \{name, changes, upload\_date, status, mod\_id\}; \\ \{name, mod\_id\} \rightarrow \{id, changes, upload\_date, status\}. \end{cases}$$

Вывод: детерминанты являются ключами. Отношение в 3НФ.

## Отношение mod\_files

Таблица 2.6 — Атрибуты отношения mod\_files

| Атрибут             | Тип                                    | Назначение            |
|---------------------|----------------------------------------|-----------------------|
| id (PK)             | Универсальный уникальный идентификатор | Идентификатор файла   |
| filename            | Строка                                 | Имя файла             |
| file_path           | Строка                                 | Путь к файлу          |
| downloads           | Целое число                            | Количество скачиваний |
| mod_version_id (FK) | Универсальный уникальный идентификатор | Ссылка на версию      |

Потенциальный ключ:  $K = \{id\}$ .

$$\Phi 3: F = \{id \rightarrow \{filename, file\_path, downloads, mod\_version\_id\}\}.$$

Вывод: поля зависят только от первичного ключа. Отношение в 3НФ.

## Отношение categories

Таблица 2.7 — Атрибуты отношения categories

| Атрибут     | Тип    | Назначение         |
|-------------|--------|--------------------|
| name (PK)   | Строка | Название категории |
| description | Текст  | Описание категории |

Потенциальный ключ:  $K = \{name\}$ .

$\Phi 3$ :  $F = \{name \rightarrow \{description\}\}$ .

Вывод: отношение находится в 3НФ.

## Отношение mods\_categories

Таблица 2.8 — Атрибуты отношения mods\_categories

| Атрибут                | Тип    | Назначение         |
|------------------------|--------|--------------------|
| mod_id (PK, FK)        | Строка | Идентификатор мода |
| category_name (PK, FK) | Строка | Название категории |

Потенциальный ключ:  $K = \{mod\_id, category\_name\}$ .

$\Phi 3$ :  $F = \emptyset$ .

Вывод: неключевые атрибуты отсутствуют. Отношение в 3НФ.

## Отношение comments

Таблица 2.9 — Атрибуты отношения comments

| Атрибут      | Тип                                    | Назначение                |
|--------------|----------------------------------------|---------------------------|
| id (PK)      | Универсальный уникальный идентификатор | Идентификатор комментария |
| content      | Текст                                  | Содержимое                |
| publish_date | Дата и время                           | Время отправки            |
| user_id (FK) | Строка                                 | Автор                     |
| mod_id (FK)  | Строка                                 | Модификация               |

Потенциальный ключ:  $K = \{id\}$ .

$\Phi 3$ :  $F = \{id \rightarrow \{content, publish\_date, user\_id, mod\_id\}\}$

*Вывод:* соблюдаются условия 3НФ.

## Отношение collections

Таблица 2.10 — Атрибуты отношения collections

| Атрибут     | Тип                                    | Назначение              |
|-------------|----------------------------------------|-------------------------|
| id (PK)     | Универсальный уникальный идентификатор | Идентификатор коллекции |
| name        | Строка                                 | Название                |
| description | Текст                                  | Описание подборки       |
| owner (FK)  | Строка                                 | Владелец                |

*Потенциальные ключи:*  $K_1 = \{id\}$ ,  $K_2 = \{name, owner\}$ .

$$\Phi 3: F = \begin{cases} id \rightarrow \{name, description, owner\}; \\ \{name, owner\} \rightarrow \{id, description\}. \end{cases}$$

*Вывод:* левые части всех ФЗ являются ключами. Отношение в 3НФ.

## Отношение collections\_mods

Таблица 2.11 — Атрибуты отношения collections\_mods

| Атрибут                | Тип                                    | Назначение              |
|------------------------|----------------------------------------|-------------------------|
| collection_id (PK, FK) | Универсальный уникальный идентификатор | Идентификатор коллекции |
| order_index (PK)       | Целое число                            | Позиция в списке        |
| mod_id (FK)            | Строка                                 | Идентификатор мода      |

*Потенциальные ключи:*  $K_1 = \{collection\_id, order\_index\}$ ,

$K_2 = \{collection\_id, mod\_id\}$ .

$$\Phi 3: F = \begin{cases} \{collection\_id, index\} \rightarrow \{mod\_id\}; \\ \{collection\_id, mod\_id\} \rightarrow \{order\_index\}. \end{cases}$$

*Вывод:* отношение находится в 3НФ.

## 2.4 Разработка алгоритма работы триггера

В системе пользователи загружают моды, которые состоят из версий и файлов. Версия мода может быть одобрена только в том случае, если к ней

прикреплен хотя бы один файл.

Чтобы не допустить ситуацию, когда версия получает статус «одобрено», но файлов у нее нет, был создан триггер. Он проверяет наличие файлов при изменении статуса версии и не дает сохранить некорректные данные.

Схема алгоритма работы триггера представлена на рисунке 2.2.

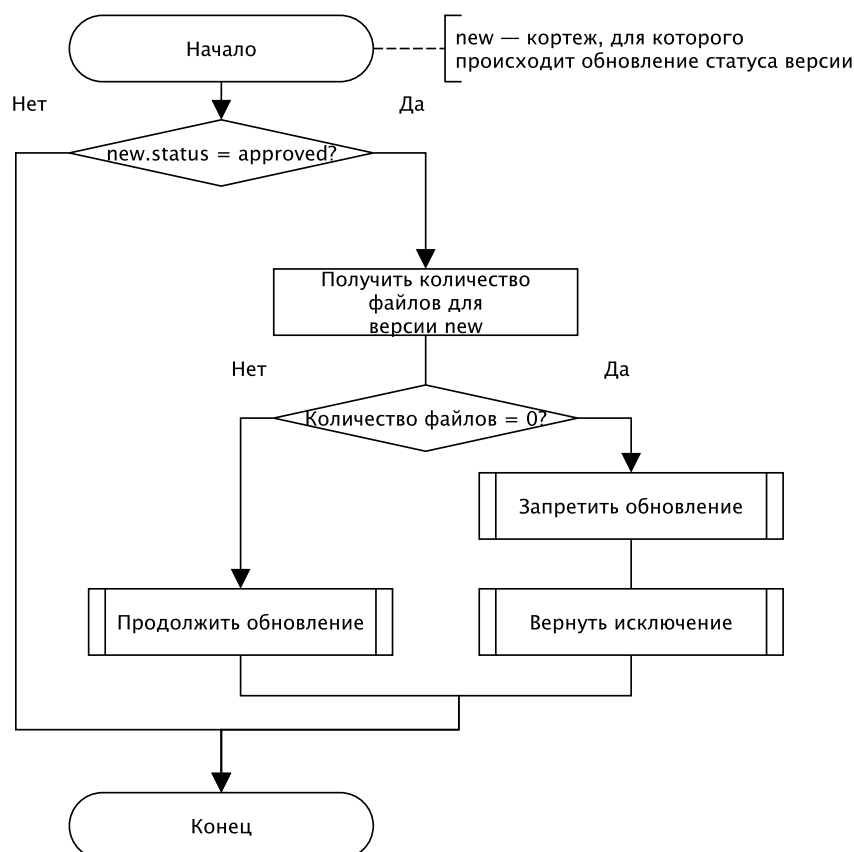


Рисунок 2.2 — Схема алгоритма работы триггера

## 2.5 Вывод

В ходе проектирования базы данных была разработана схема, состоящая из 10 взаимосвязанных таблиц. С применением матричного подхода спроектировано распределение прав доступа между тремя ролями системы. Проведенный анализ функциональных зависимостей доказал, что все отношения строго удовлетворяют требованиям третьей нормальной формы, что гарантирует целостность данных и защиту от аномалий модификации. Был спроектирован алгоритм работы триггера для реализации ограничения логики модерации.

## **3 Технологическая часть**

### **3.1 Выбор средств реализации**

В качестве системы управления базами данных была выбрана PostgreSQL [6]. Данная СУБД поддерживает реляционную модель данных, механизмы ссылочной целостности, транзакции, триггеры и индексы различных типов.

PostgreSQL предоставляет встроенные средства оптимизации SQL-запросов, поддерживает расширяемую систему типов данных и язык процедурного программирования PL/pgSQL.

Для кэширования часто используемых данных был выбран Redis [7]. Использование Redis позволяет уменьшить количество обращений к основной базе данных и сократить время выполнения запросов.

### **3.2 Реализация базы данных**

База данных была реализована в PostgreSQL в соответствии с разработанной ранее схемой.

Основными сущностями системы являются:

- пользователи;
- игры;
- модификации;
- версии модификаций;
- комментарии;
- категории;
- пользовательские подборки.

Для хранения данных использовались взаимосвязанные таблицы. Связи между таблицами реализованы с помощью внешних ключей. В качестве идентификаторов использовались первичные ключи. Для обеспечения ссылочной целостности были созданы внешние ключи. Полный SQL-скрипт создания таблиц с ограничениями приведен в листинге Б.1.

Для хранения статуса версии модификации использовался перечислимый тип представленный в листинге 3.1.

### Листинг 3.1 — Перечислимый тип статуса

```
CREATE TYPE mod_status AS ENUM (  
    'pending',  
    'approved',  
    'rejected'  
);
```

Использование перечислимого типа позволяет ограничить набор допустимых значений поля `status` и предотвратить запись некорректных данных.

В результате была реализована реляционная структура базы данных, обеспечивающая хранение взаимосвязанных данных предметной области.

## 3.3 Ограничения целостности

Для обеспечения корректности данных были реализованы ограничения целостности.

В базе данных использовались следующие типы ограничений:

- PRIMARY KEY;
- FOREIGN KEY;
- UNIQUE;
- CHECK;
- NOT NULL.

Ограничения предотвращают добавление некорректных данных и обеспечивают согласованность связанных таблиц.

Для проверки допустимых значений использовались CHECK-ограничения.

Пример ограничения представлен в листинге 3.2.

### Листинг 3.2 — CHECK-ограничение

```
downloads INTEGER NOT NULL  
CHECK(downloads >= 0)
```

Использование ограничений позволяет поддерживать корректность данных непосредственно на уровне СУБД независимо от клиентского приложения.

## 3.4 Реализация триггеров

В системе были реализованы триггеры для автоматического контроля корректности данных. Согласно требованиям предметной области, версия моди-

фикации не может получить статус «approved» без прикрепленных файлов. Для реализации данного ограничения была создана триггерная функция.

Фрагмент функции представлен в листинге 3.3.

### Листинг 3.3 — Триггерная функция

```
CREATE OR REPLACE FUNCTION
check_mod_version_files()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'approved' THEN
        IF NOT EXISTS (
            SELECT 1
            FROM mod_files
            WHERE mod_version_id = NEW.id
        ) THEN
            RAISE EXCEPTION
            'Version must contain at least one file';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

После создания функции триггер был подключен к таблице mod\_versions.

### Листинг 3.4 — Создание триггера

```
CREATE TRIGGER
mod_version_validation_trigger
BEFORE UPDATE ON mod_versions
FOR EACH ROW
EXECUTE FUNCTION
    check_mod_version_files();
```

Использование триггеров позволяет проверять корректность данных на уровне базы данных и предотвращать нарушение бизнес-логики.

## 3.5 Ролевая модель

Для разграничения доступа к объектам базы данных была реализована ролевая модель.

В системе используются следующие роли:

- mw\_guest;
- mw\_user;
- mw\_admin.

Роль mw\_guest обладает правами только на чтение данных.

Роль mw\_user может добавлять и изменять данные, связанные с пользовательским контентом.

Роль mw\_admin обладает полным доступом ко всем объектам базы данных.

Использование ролевой модели позволяет ограничить доступ пользователей к данным и уменьшить вероятность несанкционированного изменения информации.

### 3.6 Кэширование

Для уменьшения нагрузки на PostgreSQL использовался Redis.

В Redis сохранялись результаты наиболее часто выполняемых запросов:

- популярные модификации;
- результаты поиска;
- информация о пользователях.

Использование кэширования позволяет сократить количество обращений к основной базе данных и уменьшить время ответа системы при выполнении большого количества однотипных запросов, результаты которых изменяются редко.

### 3.7 Тестирование

Для проверки корректности работы базы данных было проведено тестирование ограничений целостности, триггеров и индексов.

В процессе тестирования проверялись:

- ограничения внешних ключей;
- ограничения уникальности;
- CHECK-ограничения;
- работа триггеров;
- корректность работы индексов.

Результаты тестирования триггера представлены в таблице 3.1.



Таблица 3.1 — Результаты тестирования триггера

| Тестовый сценарий                                          | Результат                    |
|------------------------------------------------------------|------------------------------|
| Изменение статуса версии на approved при наличии файлов    | Обновление успешно выполнено |
| Изменение статуса версии на pending                        | Обновление успешно выполнено |
| Изменение статуса версии на rejected                       | Обновление успешно выполнено |
| Изменение статуса версии на approved при отсутствии файлов | Сгенерирована ошибка         |

Также проводилось тестирование ограничений целостности и индексов.

Результаты тестирования показали корректную работу ограничений, триггеров, индексов и механизмов обеспечения целостности данных.

В ходе тестирования не было обнаружено нарушений ссылочной целостности или ошибок выполнения триггеров.

### 3.8 Вывод

В ходе технологической части была реализована база данных платформы игровых модификаций с использованием PostgreSQL. Была разработана реляционная структура хранения данных, реализованы ограничения целостности, триггеры, индексы и ролевая модель доступа. Использование ограничений и триггеров позволило обеспечить контроль корректности данных непосредственно на уровне СУБД. Реализация ролевой модели обеспечила разграничение доступа пользователей к объектам базы данных. Дополнительно был реализован механизм кэширования, позволяющий снизить нагрузку на PostgreSQL при выполнении часто повторяющихся запросов. Проведенное тестирование подтвердило корректность работы разработанной базы данных и реализованных механизмов обеспечения целостности данных.

## **4 Исследовательская часть**

### **4.1 Цель исследования**

При работе информационной системы поиска игровых модификаций значительная часть нагрузки приходится на выполнение поисковых запросов. По мере роста объема данных возрастает время поиска, что негативно влияет на производительность системы и пользовательский опыт.

Одним из основных механизмов повышения производительности является использование индексов. Однако различные типы индексов ориентированы на разные сценарии использования и могут демонстрировать различные результаты в зависимости от характера запросов.

Целью исследования является анализ влияния различных типов индексов PostgreSQL на скорость поиска игровых модификаций.

### **4.2 Описание исследования**

Для проведения исследования была подготовлена база данных, содержащая информацию о игровых модификациях.

Были рассмотрены следующие варианты организации хранения данных:

- отсутствие индекса;
- индекс B-Tree;
- индекс Hash;
- индекс GIN;
- индекс GiST;
- индекс BRIN.

Была составлена база данных, состоящая из 100 000 различных записей модификаций. Для каждого варианта индекса выполнялись следующие запросы:

- поиск модификации по точному совпадению значения (названию);
- поиск модификаций в заданном диапазоне значений (дата публикации);

Для повышения достоверности результатов выполнялось несколько запусков каждого запроса с последующим вычислением среднего значения.

### 4.3 Сравнение размеров индексов

Размеры созданных индексов представлены на рисунке 4.1.

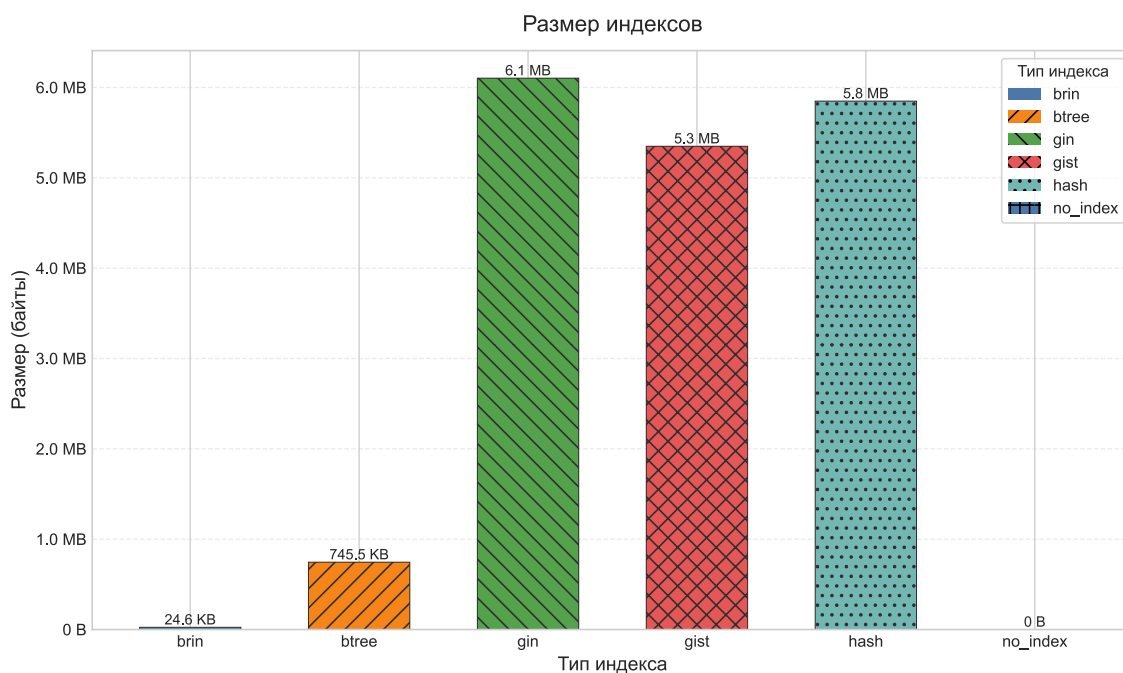


Рисунок 4.1 — Размеры исследуемых индексов

Наименьший размер продемонстрировал индекс BRIN, занимающий всего 24 КБ. Размер индекса B-Tree составил 745 КБ. Индексы Hash, GIN и GiST потребовали значительно больше дискового пространства: 5.8 МБ, 6.1 МБ и 5.3 МБ соответственно.

Таким образом, по объему занимаемой памяти наименьшим является индекс BRIN, тогда как индексы GIN и Hash требуют более чем в 200 раз больше памяти.

## 4.4 Исследование скорости поиска по точному совпадению

Результаты измерений представлены на рисунке 4.2.

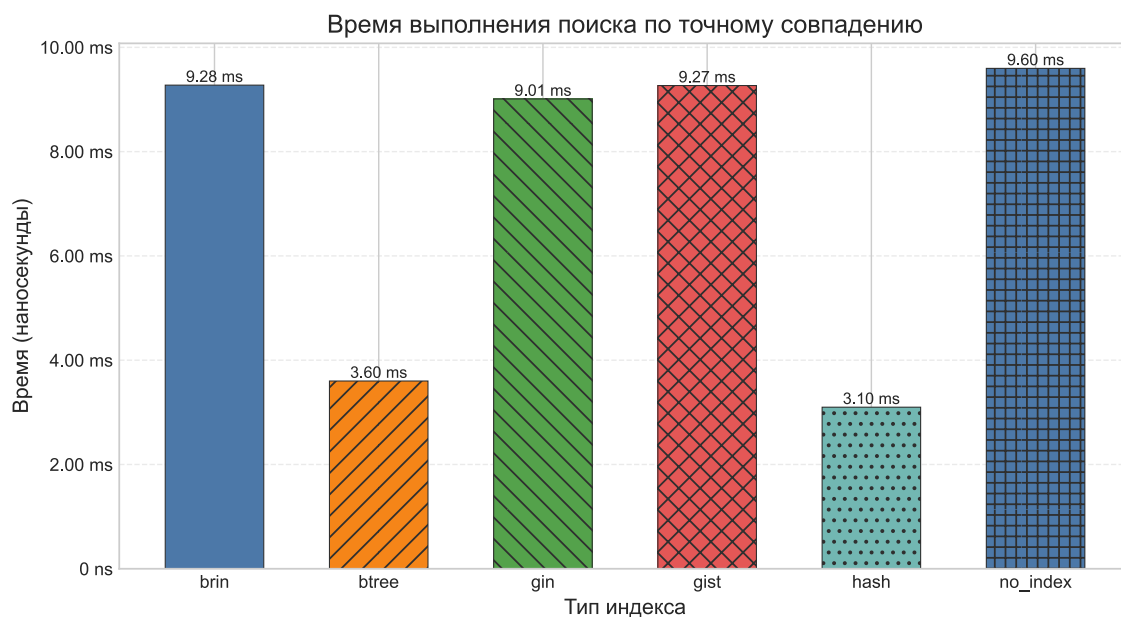


Рисунок 4.2 — Время поиска по точному совпадению

Результаты исследования представлены в таблице 4.1.

Таблица 4.1 — Время поиска по точному совпадению

| Тип индекса | Время, нс |
|-------------|-----------|
| Без индекса | 9 596 800 |
| B-Tree      | 3 600 858 |
| Hash        | 3 098 758 |
| GIN         | 9 009 500 |
| GiST        | 9 267 383 |
| BRIN        | 9 275 283 |

Наилучший результат продемонстрировал индекс Hash, обеспечивший выполнение запроса за 3.1 миллисекунды. По сравнению с отсутствием индекса ускорение составило примерно 68%.

Индекс B-Tree показал сопоставимый результат — 3.6 миллисекунды, что соответствует ускорению на 62.5%.

Индексы GIN, GiST и BRIN практически не повлияли на производительность поиска по точному совпадению. Время выполнения запросов оказалось близким к варианту без индекса.

Полученные результаты объясняются тем, что Hash и B-Tree специально оптимизированы для операций сравнения по равенству.

## 4.5 Исследование диапазонного поиска

Результаты диапазонного поиска представлены на рисунке 4.3.

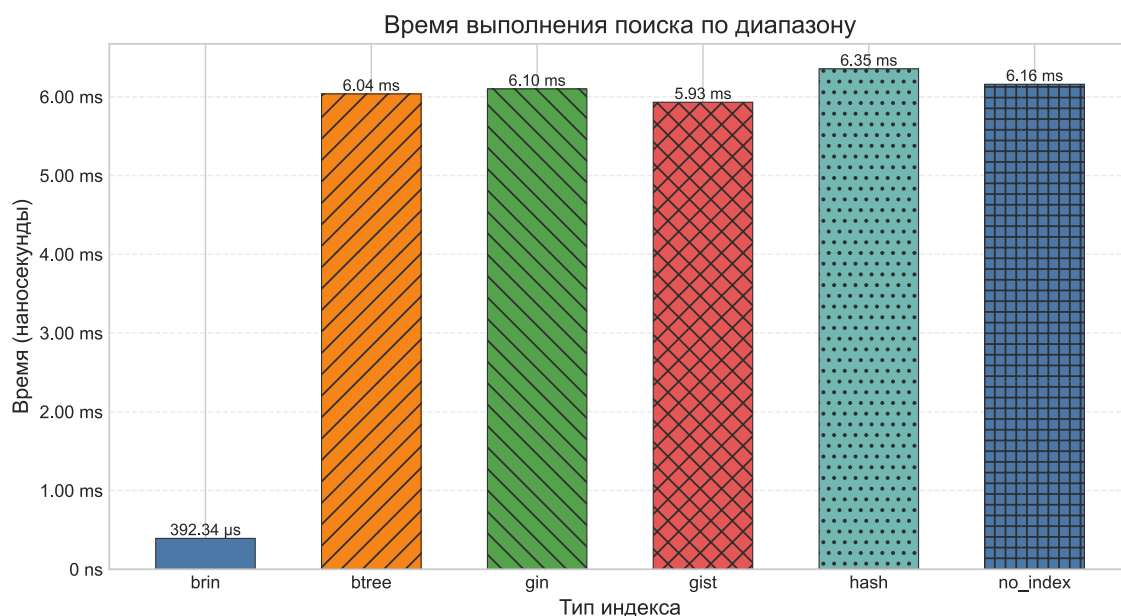


Рисунок 4.3 — Время диапазонного поиска

Результаты измерений приведены в таблице 4.2.

Таблица 4.2 — Время диапазонного поиска

| Тип индекса | Время, нс |
|-------------|-----------|
| Без индекса | 6 159 675 |
| B-Tree      | 6 036 067 |
| Hash        | 6 354 733 |
| GIN         | 6 099 567 |
| GiST        | 5 931 292 |
| BRIN        | 392 342   |

Самым быстрым оказался индекс BRIN. Время выполнения запроса составило всего 392 тыс. нс, что в 15.7 раза быстрее последовательного сканирования таблицы.

Остальные типы индексов показали результаты, близкие друг к другу. Улучшение производительности относительно отсутствия индекса оказалось

незначительным.

Высокая скорость BRIN объясняется тем, что данный индекс хранит сведения о диапазонах значений и позволяет быстро исключать большие фрагменты таблицы, не содержащие подходящих записей.

## 4.6 Вывод

Проведенное исследование показало, что скорость поиска при наличии индексов PostgreSQL существенно зависит от характера выполняемых запросов.

Для поиска игровых модификаций по точному совпадению наилучшие результаты продемонстрировали индексы Hash и B-Tree. Минимальное время выполнения запроса было получено при использовании Hash-индекса и составило 3.1 миллисекунды, что примерно в 3.1 раза быстрее по сравнению с поиском без индекса. Однако размер Hash-индекса составил 5.8 МБ, что почти в восемь раз превышает размер индекса B-Tree. Индекс B-Tree показал близкий результат — 3.6 миллисекунды, обеспечив ускорение поиска более чем в 2.5 раза при значительно меньших затратах памяти. Это позволяет рассматривать B-Tree как лучшее решение для реализации поиска модификаций по наименованию среди других индексов.

При выполнении диапазонных запросов наилучший результат продемонстрировал индекс BRIN. Время выполнения запроса составило 392 тыс. нс, что в 15.7 раза быстрее по сравнению с последовательным сканированием таблицы. Дополнительным преимуществом данного индекса является минимальный объем занимаемой памяти — 24 КБ. Полученные результаты подтверждают высокую скорость BRIN для работы с большими объемами данных, имеющими естественную упорядоченность значений.

Таким образом, результаты исследования позволяют сделать вывод, что для разработанной базы данных поиска игровых модификаций наиболее рациональным решением является использование индекса B-Tree в качестве основного механизма индексирования. Данный тип индекса обеспечивает высокую скорость поиска при умеренных затратах памяти и является наиболее универсальным для выполнения типовых запросов. Для операций диапазонного поиска целесообразно дополнительно использовать индекс BRIN, который обеспечивает максимальную скорость поиска при минимальном размере индекса.

# ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы была разработана база данных для платформы игровых модификаций, предназначенная для хранения, управления и поиска информации о модификациях, их версиях, файлах, пользователях, комментариях и пользовательских подборках.

В ходе работы проведен анализ предметной области, выделены основные сущности системы и сформулированы требования к разрабатываемому решению. Выполнено сравнение различных моделей хранения данных, по результатам которого была выбрана реляционная модель, обеспечивающая необходимый уровень целостности данных и поддержку сложных связей между объектами предметной области.

Спроектирована структура базы данных, разработана диаграмма «сущность–связь» и выполнена нормализация отношений до третьей нормальной формы. Реализована ролевая модель доступа, позволяющая разграничить права пользователей различных категорий. Для контроля корректности данных создан триггер, обеспечивающий соблюдение требований бизнес-логики.

Разработанная база данных реализована в СУБД PostgreSQL с использованием ограничений целостности, индексов и триггеров. Для повышения производительности системы предусмотрено использование Redis в качестве средства кэширования часто запрашиваемых данных.

Проведенное исследование показало, что производительность индексирования зависит от характера выполняемых запросов. Наиболее универсальным решением для разработанной системы является использование индексов типа B-Tree, обеспечивающих хорошее соотношение между скоростью поиска и объемом занимаемой памяти. Для диапазонного поиска целесообразно дополнительно применять индексы BRIN.

Поставленная цель работы достигнута, а все поставленные задачи успешно выполнены.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Chen P. P. S. The Entity-Relationship Model — Toward a Unified View of Data // ACM Transactions on Database Systems. — 1976. — Т. 1. — №. 1. — с. 9–36.
2. Аврунев, О. Е. Модели баз данных : учебное пособие / О. Е. Аврунев, В. М. Стасышин. — Новосибирск : Изд-во НГТУ, 2018. — 124 с.
3. Codd, E. F. A Relational Model of Data for Large Shared Data Banks / E. F. Codd // Communications of the ACM. — 1970. — Т. 13 — №. 6. — с. 377–387.
4. Основы баз данных: учебное пособие / С.Д. Кузнецов — 2-е изд., испр. — М.: Интернет-Университет Информационных Технологий; БИНОМ. Лаборатория знаний, 2007. — 484 с.
5. Путеводитель по базам данных. / В.И. Комаров — М.: ДМК-Пресс, 2024. — 520 с.
6. Документация СУБД PostgreSQL // URL: <https://www.postgresql.org/docs/>
7. Документация In-Memory БД Redis // URL: <https://redis.io/docs/latest/>



# Приложение А

К РПЗ работы прилагаются следующие материалы:

- 1) презентация к курсовой работе, состоящая из 17 слайдов.
- 2) USB флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации. Структура файлов на накопителе:

```
|— report
|   |— Презентация.pdf
|   |— РПЗ.pdf
|— code ..... исходный код проекта
|   |— docker ... ..... файлы для развертывания контейнера
|   |— sql ... ..... SQL скрипты
|   |— gradle ... ..... дополнительные скрипты системы сборки
|   |— layers..... слои чистой архитектуры
|       |— data-access ... ..... исходный код доступа к данным
|       |— business-logic ... ..... исходный код бизнес логики
|       |— ui ... ..... исходный код контроллеров
|   |— src ... ..... исходный код точки входа в приложение
|   |— gradle.build..... главный скрипт системы сборки
|   |— settings.gradle..... конфигурационный файл сборки
```

# Приложение Б

## Листинг Б.1 — SQL-скрипт создания таблиц

```
drop schema if exists modweave cascade;
create schema modweave;

create table modweave.users (
    username varchar primary key,
    name varchar unique not null,
    email varchar unique not null,
    password varchar not null,
    register_date timestamp default current_timestamp not null,
    is_admin boolean
);

create table modweave.games (
    id varchar primary key,
    name varchar not null,
    description text,
    image_path varchar not null
);

create table modweave.mods (
    id varchar primary key,
    name varchar not null,
    description text,
    image_path varchar not null,
    creation_date timestamp default current_timestamp not null,
    game_id varchar not null references modweave.games (id) on
        delete cascade,
    publisher_id varchar not null references modweave.users (
        username
    ) on delete cascade
);

create type modweave.version_status as enum ('PENDING', '
    APPROVED', 'REJECTED');

create table modweave.mod_versions (
    id uuid primary key,
```

```

name varchar not null,
changes text,
upload_date timestamp default current_timestamp not null,
status modweave.version_status default 'PENDING',
mod_id varchar not null references modweave.mods (id) on delete
    cascade,
unique (name, mod_id)
) ;

create table modweave.mod_files (
id bigserial primary key,
filename varchar not null,
file_path varchar not null,
downloads int check (downloads >= 0),
mod_version_id uuid not null references modweave.mod_versions (
    id) on delete cascade
) ;

create table modweave.categories (
name varchar primary key,
description text
) ;

create table modweave.mods_categories (
mod_id varchar not null references modweave.mods (id) on delete
    cascade,
category_name varchar references modweave.categories (name) on
    delete cascade,
primary key (mod_id, category_name)
) ;

create table modweave.comments (
id bigserial primary key,
content text not null,
publish_date timestamp default current_timestamp not null,
user_id varchar not null references modweave.users (username)
    on delete cascade,
mod_id varchar not null references modweave.mods (id) on delete
    cascade,

```

```

parent_comment_id bigint references modweave.comments (id) on
    delete cascade
) ;

create table modweave.collections (
id bigserial primary key,
name varchar not null,
description text,
owner varchar not null references modweave.users (username) on
    delete cascade,
unique (name, owner)
) ;

create table modweave.collections_mods (
collection_id bigint not null references modweave.collections (
    id) on delete cascade,
order_index int not null,
mod_id varchar not null references modweave.mods (id) on delete
    cascade,
primary key (collection_id, order_index),
unique (collection_id, mod_id)
) ;

```

## Листинг Б.2 — SQL-скрипт создания ролей

```

drop role if exists mw_guest;
drop role if exists mw_user;
drop role if exists mw_admin;

create role mw_guest;
create role mw_user;
create role mw_admin;

grant usage on schema modweave to mw_guest, mw_user, mw_admin;

grant select on table modweave.users to mw_guest;
grant select, update on table modweave.users to mw_user;
grant all privileges on table modweave.users to mw_admin;

grant select on table modweave.games to mw_guest;
grant select on table modweave.games to mw_user;

```

```

grant all privileges on table modweave.games to mw_admin;

grant select on table modweave.mods to mw_guest;
grant all privileges on table modweave.mods to mw_user;
grant all privileges on table modweave.mods to mw_admin;

grant select on table modweave.mod_versions to mw_guest;
grant all privileges on table modweave.mod_versions to mw_user;
grant all privileges on table modweave.mod_versions to mw_admin
    ;

grant select on table modweave.mod_files to mw_guest;
grant select, insert, delete on table modweave.mod_files to
    mw_user;
grant all privileges on table modweave.mod_files to mw_admin;

grant select on table modweave.categories to mw_guest;
grant select on table modweave.categories to mw_user;
grant all privileges on table modweave.categories to mw_admin;

grant select on table modweave.mods_categories to mw_guest;
grant select, insert, delete on table modweave.mods_categories
    to mw_user;
grant all privileges on table modweave.mods_categories to
    mw_admin;

grant select on table modweave.comments to mw_guest;
grant select, insert, delete on table modweave.comments to
    mw_user;
grant all privileges on table modweave.comments to mw_admin;

grant select on table modweave.collections to mw_guest;
grant all privileges on table modweave.collections to mw_user;
grant all privileges on table modweave.collections to mw_admin;

grant select on table modweave.collections_mods to mw_guest;
grant all privileges on table modweave.collections_mods to
    mw_user;
grant all privileges on table modweave.collections_mods to
    mw_admin;

```

```
grant usage, select on sequence modweave.mod_files_id_seq to
    mw_user, mw_admin;
grant usage, select on sequence modweave.comments_id_seq to
    mw_user, mw_admin;
grant usage,
select on sequence modweave.collections_id_seq to mw_user,
mw_admin;
```